Project - Low Level Design

On

Telecomm Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ANKIT YADAV	EN22CS301136
2	ANSHUMAN GEETE	EN22CS301160
3	ARTI JAIN	EN22CS301206
4	ATISHA JAIN	EN22CS301235
5	AVANI SHARMA	EN22CS301239
6	ISHANA HATODIYA	EN23CS3L1011

Group Name: 07D2

Project Number: AAI-19

Industry Mentor Name:

University Mentor Name:

Academic Year: 2022-2026

1. Introduction :

The Telecom Support Triage Agent is an AI-driven system designed to automate the handling of customer support queries in telecom services. The system leverages Agentic AI architecture using multiple intelligent agents to:

- Understand customer queries
- Classify issues, Assign priority
- Route tickets to the appropriate team
- Provide automated resolutions when possible

1.1. Scope of the document :

- Internal system architecture
- Component-level behavior
- Data and control flow
- API specifications
- Design considerations
- It focuses on implementation-level details for developers and engineers.

1.2. Intended Audience :

- Backend Developers
- AI/ML Engineers
- Data Scientists
- System Architects
- QA Engineers
- DevOps Engineers

1.3. System overview :

The system acts as an intelligent triage assistant for telecom queries.

Core Features:

- Natural Language Understanding (NLU)
- Issue classification (Billing, Network, SIM, etc.)
- Priority detection (High/Medium/Low)
- Automated response generation

Tech Stack:

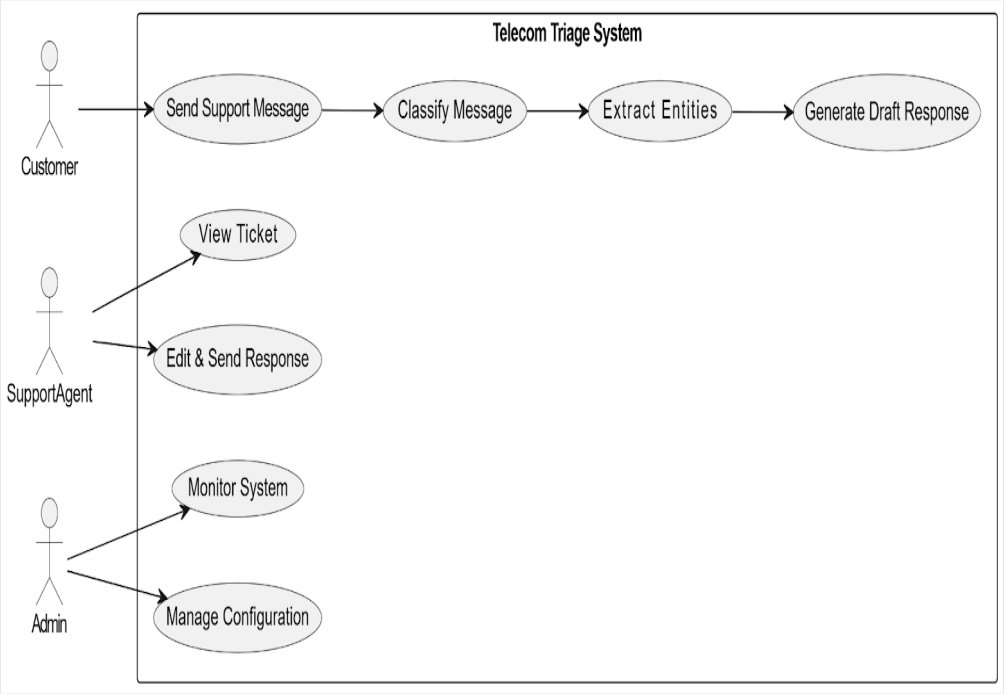
- Python (Core backend)
- LangChain / CrewAI (Agent orchestration)
- LLM API (OpenAI / Groq / Claude)
- Vector DB (FAISS/Pinecone for knowledge retrieval)

2. System Design :

2.1. Use Case Diagram:

This diagram shows how the Telecom Triage System works:

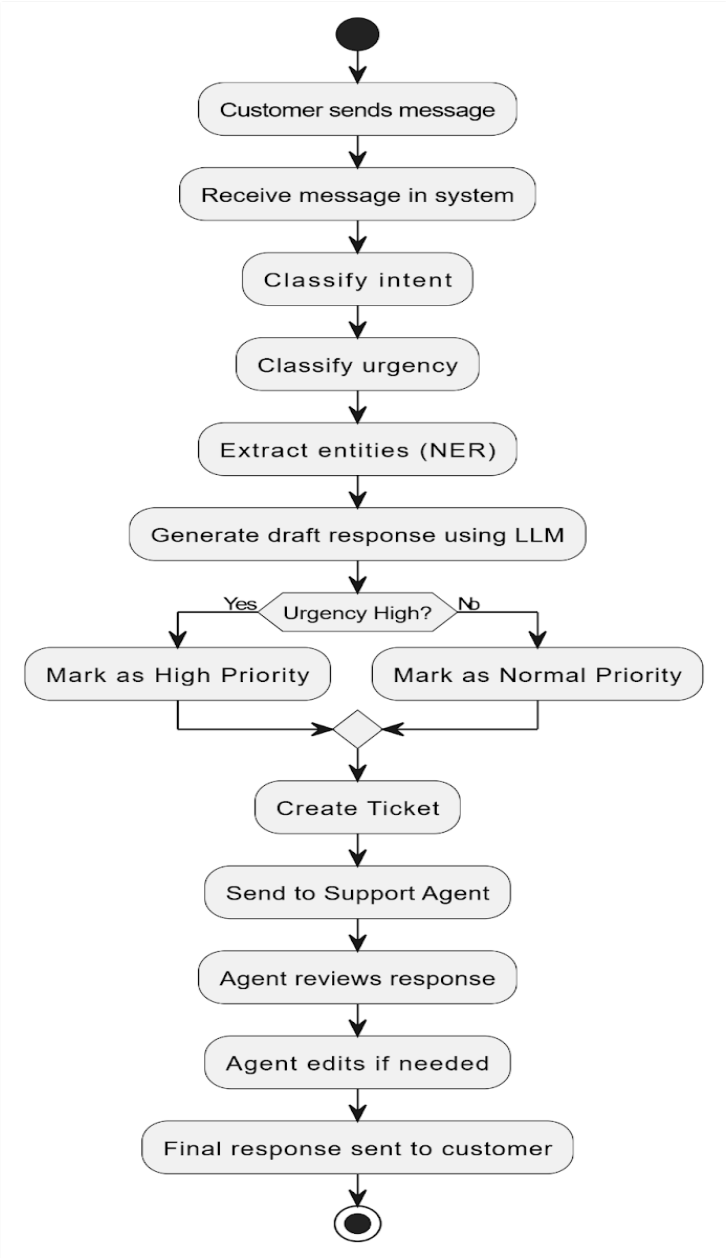
A customer sends a support message, the system processes it (classifies, extracts details, generates a response), and a support agent reviews and sends the final reply, while the admin manages and monitors the system.



2.2. Process Flow :

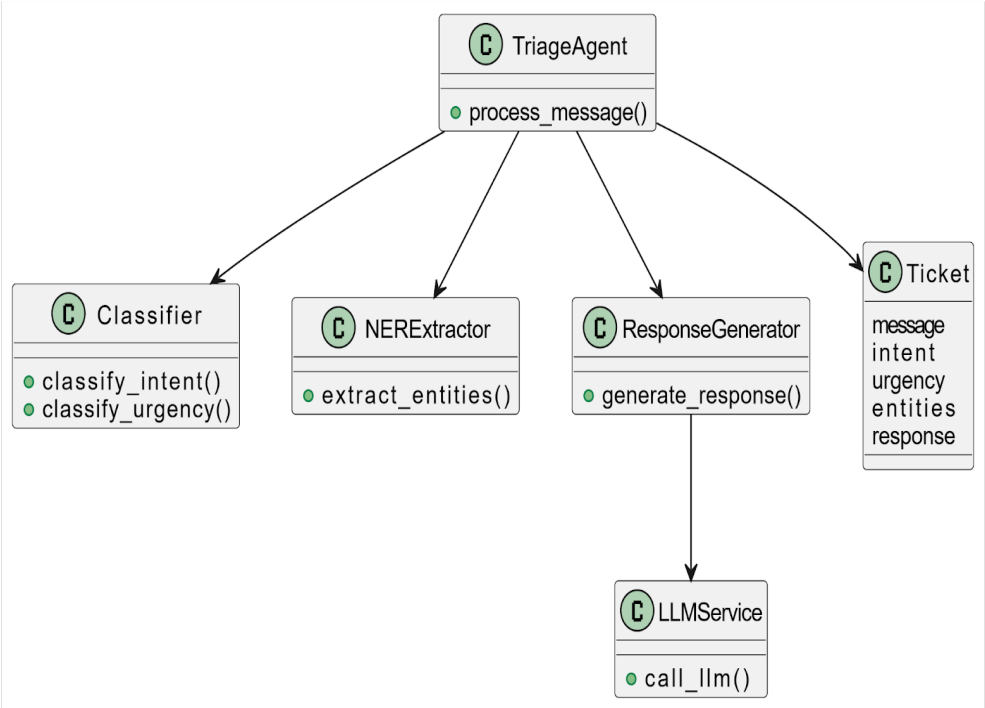
This activity diagram shows the flow of the Telecom Triage System:

A customer message is analyzed (intent, urgency, entities), a draft response is generated, priority is assigned, a ticket is created, and the support agent reviews and sends the final reply.



2.3. Class Diagram :

- The class diagram shows the structure of the Telecomm Triage System.
- Triage Agent is the main class that processes incoming messages.
- It uses the Classifier to identify intent and urgency.
- The NER Extractor extracts important entities from the message.
- The Response Generator creates replies using the LLM Service.
- All processed information is stored in the Ticket class.
- This design separates tasks into modules for better organization and scalability.

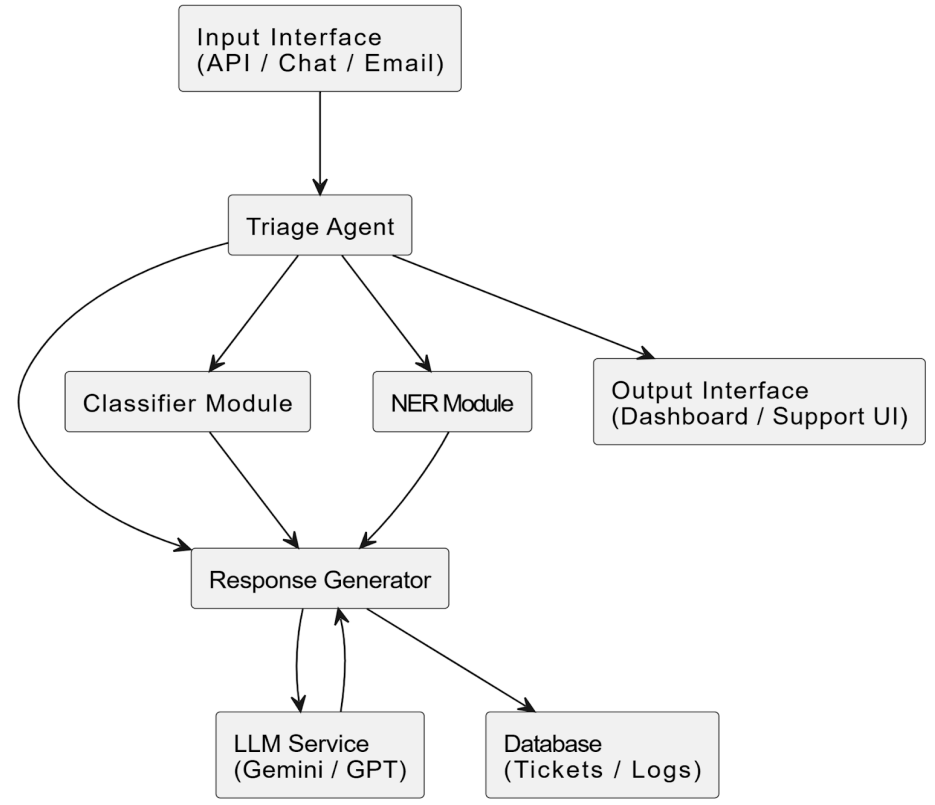


2.4. Components Design :

The diagram shows the architecture of the Telecom Triage System.

- The Input Interface (API/Chat/Email) receives user queries.
- These queries are handled by the Triage Agent, which controls the workflow.
- It uses the Classifier Module to detect intent and urgency.

- The NER Module extracts important entities from the message.
- The Response Generator creates replies using the LLM Service.
- Finally, results are stored in the Database and shown via the Output Interface.

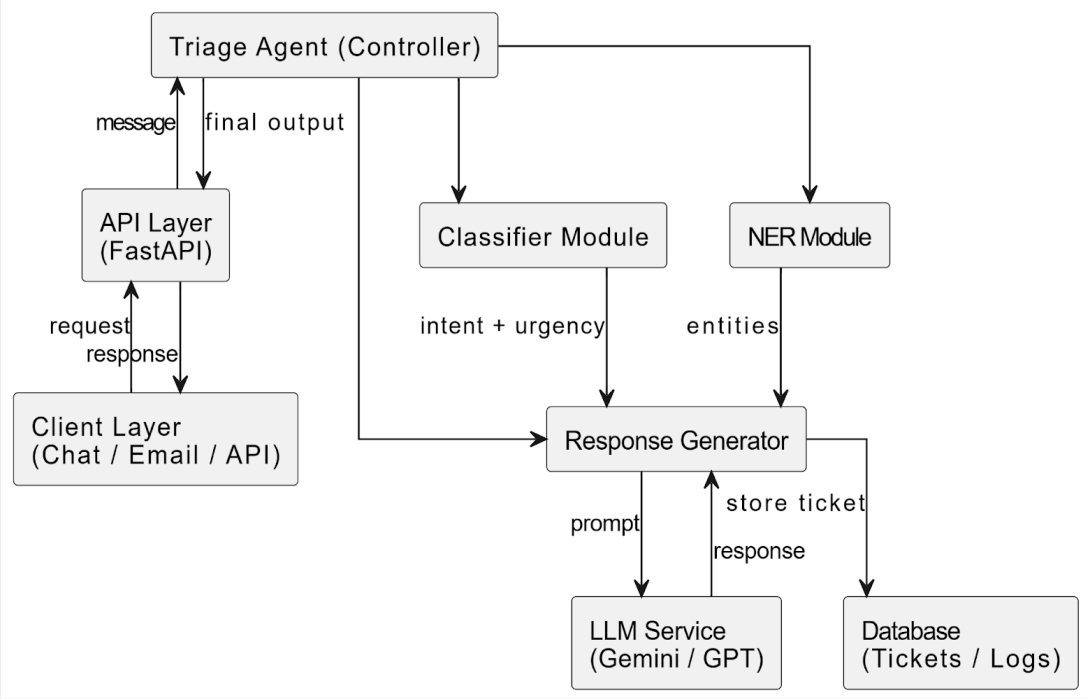


2.5. Architecture Diagram :

The diagram shows the overall architecture of the Telecom Triage System.

- The Client Layer (chat/email/API) sends user queries to the API Layer (FastAPI).
- The API forwards the request to the Triage Agent, which acts as the controller.
- The Triage Agent sends data to the Classifier Module for intent and urgency detection.
- It also sends data to the NER Module to extract key entities.
- These outputs are passed to the Response Generator to create a reply.

- The Response Generator uses the LLM Service (Gemini/GPT) to generate responses.
- Finally, responses are stored in the Database and sent back to the client.



2.6. Key Design Consideration :

1. Performance :

- Use caching (Redis)
- Async LLM calls

2. Security :

- JWT authentication
- Input sanitization

3. Scalability :

- Stateless services
- LLM Optimization

- Prompt tuning
- Few-shot learning
- Token cost optimization

4.Accuracy :

Combine LLM + rule-based validation

5.Fault Tolerance :

- Retry mechanism
- Graceful fallback

2.7. API Catalogue :

The Gemini/GPT API is used to perform the following tasks:

- Intent classification of customer queries
- Urgency detection
- Named Entity Recognition (NER)
- Draft response generation

2.8. Technology Stack :

The system is built using the following technology stack:

- **Python** – Core backend development and system logic
- **FastAPI** – For building lightweight and high-performance APIs
- **LangChain / CrewAI** – For implementing Agentic AI workflows and orchestration
- **Gemini API (LLM)** – For intent classification, entity extraction, and response generation.

- **HuggingFace** – For additional Named Entity Recognition support
- **FAISS** – For vector-based knowledge retrieval
- **ChromaDB** – For storing tickets and logs

3. References :

- LangChain Documentation
- CrewAI Documentation
- OpenAI / Groq / Claude API Docs
- FAISS / Pinecone Documentation
- REST API Design Best Practices